

become.dev — EPIC Backlog

Frontend Engineering Learning Platform · 16 EPICs · Jira-style · v1.4 · June 2026

Changes from v1.3: added EPIC-16 (The War Room, P2, L) — formalizes the incident-style challenge lifecycle (briefing → investigation → post-mortem) as a timed IDENTIFY→FIX chain on top of the EPIC-08 exercise engine; no hints by design, itemized payout, paid tier only. Changes from v1.2: EPIC-05 redesigned — onboarding runs BEFORE auth with an experience fork ('Just starting' goes straight to Path 1 with no assessment; 'Dev' takes the 6-question assessment); signup moves to the recommendation step (peak commitment); post-signup lands in lesson 1, not the dashboard; EPIC-02 gains a ?next=intent-param redirect. Changes from v1.1: free/paid boundary DECIDED — Option B: Foundations track completely free; Professional + Advanced paid (track-level entitlements). Phase 1 paid product is now **Professional early access**; Foundations free at launch as acquisition engine and platform proof. Free extras defined: first lesson of every paid module (freePreview flag), curated Field Notes subset, Interview Accelerator teaser. EPIC-03 pre-sale copy, EPIC-04 entitlement states, EPIC-07 lock model and EPIC-14 updated accordingly. Carried from v1.1: reward-currency cookie economy, corrected EPIC-07/11 dependency, interim repo-based IMPLEMENT, EPIC-14/15 additions.

#	Title	Priority	Estimate	Status	Key Dependency
EPIC-01	Design System & UI Components	P0	L	Backlog	None
EPIC-02	Auth & User Identity	P0	M	Backlog	EPIC-01
EPIC-03	Landing Page	P1	M	✓ Done*	EPIC-01
EPIC-04	Catalog Page	P1	M	Backlog	EPIC-01, 02, 14
EPIC-05	Onboarding	P1	M	Backlog	EPIC-01, 02
EPIC-06	Dashboard & Progress Tracking	P1	L	Backlog	EPIC-01, 02, 08
EPIC-07	Module Engine	P0	XL	In progress	EPIC-01, 02, 11
EPIC-08	Exercise & Cookie System	P0	XL	Backlog	EPIC-07
EPIC-09	Final Assessment & Module Completion	P1	L	Backlog	EPIC-07, 08
EPIC-10	Field Notes	P2	M	Backlog	EPIC-07, 09
EPIC-11	Tracks & Content Structure	P0	L	✓ Done	None
EPIC-12	In-Browser Code Editor & Test Runner	P2	XL	Backlog	EPIC-08
EPIC-13	Interview Accelerator	P2	XL	Backlog	EPIC-07, 11
EPIC-14	Payments & Entitlements (Option B decided)	P0	M	Backlog	EPIC-02
EPIC-15	Launch Readiness (email, analytics, legal)	P1	M	Backlog	EPIC-02, 14
EPIC-16	The War Room	P2	L	Backlog	EPIC-08, 09, 14

Estimate: S = 1-3 days · M = 1-2 weeks · L = 2-4 weeks · XL = 4-6 weeks | Priority: P0 = must have before launch · P1 = core experience · P2 = enhances experience | *EPIC-03: core page shipped; waitlist/pre-sale follow-up pending.

Launch Sequencing (v1.4)

Phase	Order
Phase 1 — Sellable Slice	EPIC-11 (done) → EPIC-03 pre-sale follow-up ('Foundations free at launch, lock the Professional early-access price') → EPIC-01 (scoped) → EPIC-02 → EPIC-07 (in progress) → EPIC-08 → EPIC-14 → content gate: ≥3 Foundations modules + ≥1 Professional module → LAUNCH: Foundations FREE + Professional PAID early access
Phase 2 — Complete Core	EPIC-09 (fast-follow) → EPIC-15 (email required once money moves) → EPIC-05 → EPIC-06 → EPIC-04 (Option B entitlement badges)

Phase 3 — Post-revenue, demand-driven	EPIC-13 (free teaser ships earlier; full or standalone product) → EPIC-16 (war room, retention for paid users) → EPIC-10 (free subset ships with launch) → EPIC-12 (upgrades IMPLEMENT and war room fix step)
--	---

Content production via the multi-agent pipeline runs in parallel through every phase and is a formal launch gate: Foundations content (currently 2/7) unblocks the free launch; Professional content (currently 0/9) unblocks revenue. Content is the critical path; the platform never blocks it.

EPIC-01 — Design System & UI Components

Status: **Backlog** · Priority: **P0** · Estimate: **L** · Depends on: **None**

Description	Build the shared component library that the entire platform is built on top of. Documented with Storybook. Every other EPIC depends on this being in place first. v1.1 : scope the first pass to components needed by the Phase 1 sellable slice (Module Engine, Exercise system, Auth, Checkout); remaining components land on demand.
Deliverables	• Storybook instance with all components documented • Tailwind config with design tokens (colours, spacing, typography) • Core UI components: Button, Badge, Card, Modal, Input, Tabs, ProgressBar, Toast, Tooltip • Layout components: Topbar, Sidebar, MobileDrawer • Dark theme implemented as default
Acceptance Criteria	• All components render correctly at mobile (375px) and desktop (1440px) • Every component has at least one Storybook story • Tailwind tokens documented in Storybook • No component has hardcoded colour values outside the token system
Out of Scope	• Page-level layouts • Feature-specific components (exercise blocks, quiz cards) • Light theme (deferred)
Dependencies	• None — this is the foundation
Initial Tasks	• Initialise Next.js project with Tailwind • Configure Storybook • Define design token system (colours, spacing, type scale) • Build Button variants • Build Badge and Tag components • Build Card component • Build Modal • Build Input and Form elements • Build Tabs component • Build ProgressBar • Build Toast notification • Build Topbar layout component • Build Sidebar layout component • Build MobileDrawer component • Write Storybook stories for all components
Resources	• Tailwind CSS docs • Storybook docs • Radix UI primitives (recommended for accessible base components)

EPIC-02 — Auth & User Identity

Status: **Backlog** · Priority: **P0** · Estimate: **M** · Depends on: **EPIC-01**

Description	Implement user registration, login, session management, and basic profile storage. This is infrastructure — no user-facing feature that persists state can exist without it.
Deliverables	• Sign up and Login pages • Session management via Supabase Auth • Protected route middleware for platform pages • Basic user profile stored in DB (name, email, chosen path, created_at) • Auth state accessible globally via context and hook • ?next= intent-param redirect after signup/login (allowlisted internal paths only)
Acceptance Criteria	• Unauthenticated users are redirected to login when accessing protected routes • Session persists across page refreshes • Logout clears session completely • Auth errors surface clearly to the user • Post-auth redirect honours ?next= for internal paths and never allows open redirects
Out of Scope	• OAuth and social login (deferred) • Email verification flow (deferred) • Password reset (deferred) • User profile editing page
Dependencies	• EPIC-01 — needs UI components for forms and pages
Initial Tasks	• Set up Supabase project • Configure Supabase Auth in Next.js • Build Sign Up page • Build Login page • Implement protected route middleware • Create useAuth hook • Set up users table in Supabase • Store basic profile on registration • Implement ?next= redirect with internal-path allowlist • Handle auth error states in UI
Resources	• Supabase Auth docs • Next.js middleware docs

EPIC-03 — Landing Page

Status: **Done** · Priority: **P1** · Estimate: **M** · Depends on: **EPIC-01**

Description	Public-facing promotional page. Sells the platform, describes what it is and who it is for, and drives users toward sign up. SEO-indexed. No auth required. v1.2: core page shipped. A small follow-up adds the waitlist + pre-sale block with the decided positioning: 'Foundations free at launch — lock the early-access price for the Professional track.' Waitlist signups size the free funnel; pre-orders validate willingness to pay for Professional. Do this before any other Phase 1 work continues.
Deliverables	• Hero section with value proposition and CTA — SHIPPED • Tracks overview, features, social proof placeholder, footer — SHIPPED • OG meta tags, responsive at all breakpoints — SHIPPED • NEW: waitlist email capture wired to provider • NEW: Professional early-access pre-sale block ('lock the price') with checkout link or 'reserve' flow • NEW: 'Foundations is free' messaging in hero / tracks section
Acceptance Criteria	• Page loads under 2s on a standard connection — MET • Lighthouse accessibility 90+ — MET • NEW: waitlist signups stored and exportable • NEW: pre-sale conversion measurable (visits → signups → pre-orders)
Out of Scope	• Actual testimonials (placeholder only) • Blog or resources section • Full pricing page (single early-access offer only)
Dependencies	• EPIC-01 — Design System (shipped with interim styles; reconcile tokens later)
Initial Tasks	• Add waitlist capture section • Wire to email provider audience • Add Professional early-access pre-sale block with working hypothesis price • Update hero/tracks copy: Foundations free at launch • Add basic conversion tracking events • Announce on existing channels (YouTube / socials)
Resources	• Next.js metadata API docs

EPIC-04 — Catalog Page

Status: **Backlog** · Priority: **P1** · Estimate: **M** · Depends on: **EPIC-01, EPIC-02**

Description	Public browsable page showing all tracks and modules. No auth required. Users can preview module content. Clicking to start a module triggers redirect to sign up. Initially hardcoded, later wired to content layer. v1.2: boundary decided (Option B): Foundations modules show FREE; Professional/Advanced modules show paid state with 'first lesson free' preview affordance.
Deliverables	• Track listing with all 3 paths visible • Module cards with title, description, lesson count, difficulty • Free / paid state visible per module (Option B: Foundations FREE; Professional/Advanced PAID with first-lesson preview badge) • Module preview state (read-only, no interaction) • Sign up prompt triggered on any interaction CTA • Hardcoded data in first iteration, content-layer driven later
Acceptance Criteria	• All 3 tracks and their modules are visible without login • Module cards show correct metadata and entitlement state • Clicking Start Module redirects unauthenticated users to Sign Up • Page is SEO-indexed with correct meta per module
Out of Scope	• Filtering or search (deferred) • User progress indicators (requires auth) • Real content data (wired via EPIC-11 loader)
Dependencies	• EPIC-01 — Design System • EPIC-02 — Auth (for redirect on CTA) • EPIC-14 — entitlement model (track-level + freePreview flag)
Initial Tasks	• Define hardcoded catalog data structure • Build Track section component • Build Module Card component (incl. entitlement badge) • Build Module Preview page (read-only) • Implement auth-wall redirect on CTA • Add SEO meta per module page • Responsive QA

EPIC-05 — Onboarding

Status: **Backlog** · Priority: **P1** · Estimate: **M** · Depends on: **EPIC-01, EPIC-02**

Description	Onboarding runs BEFORE auth (v1.3 redesign). First step is an experience fork: 'Just starting' goes straight to signup with Path 1 (Zero to Junior) preselected — no assessment, a code quiz would be a failure experience for true beginners. 'Dev — assess my level first' leads to a 5-6 question pre-auth assessment (stateless, sessionStorage) and a track recommendation with manual override. Signup happens AT the recommendation step — peak commitment — and the chosen path persists to the profile on registration. First login lands directly in lesson 1 of the chosen path, never the empty dashboard. The fork answer doubles as zero-cost audience segmentation (beginner vs dev share, see EPIC-15 events).
Deliverables	• Experience fork screen: 'Just starting' / 'Dev — assess my level first' • 'Just starting' branch: direct signup handoff with Path 1 preselected • Pre-auth multi-step assessment UI (no account required; state in sessionStorage) • Scoring logic mapping answers to track recommendation • Track recommendation screen with explanation and manual override • 'I already know my path' skip link to manual track picker • Signup handoff carrying chosen path + intent param (?next=lesson URL) • Path and assessment answers persisted to profile on registration • Post-signup redirect to lesson 1 of the chosen path
Acceptance Criteria	• Fork is the first onboarding step; both options reachable in one tap • 'Just starting' users never see an assessment question; Path 1 is preset at signup • Assessment is fully usable without an account and completes in under 3 minutes • Every answer combination produces a valid track recommendation • User can change their recommended track before confirming • Chosen path persists to DB on registration and is reflected in Dashboard • First login after onboarding lands in lesson 1 of the chosen path, not the dashboard • Abandoning mid-assessment restarts it on return (documented, accepted: <3 min flow)
Out of Scope	• Re-taking the assessment later (deferred) • Detailed skill breakdown report • Saving partial anonymous assessment progress across sessions
Dependencies	• EPIC-01 — Design System • EPIC-02 — Auth, only for the signup handoff and persistence (the quiz itself is pre-auth) • EPIC-03 — landing CTA wiring ('choose your path' routes to the fork)
Initial Tasks	• Build experience fork screen (two options) • Define assessment questions and scoring matrix • Build pre-auth multi-step form component with sessionStorage state • Implement scoring logic • Build recommendation screen • Build manual track selector + 'I already know my path' skip link • Implement signup handoff (path + ?next= intent param) • Persist path and answers to profile on registration • Implement post-signup redirect to lesson 1 • Fire segmentation events (fork choice, assessment completion) — EPIC-15 taxonomy • Handle edge cases (all equal scores, back navigation, refresh mid-quiz)

EPIC-06 — Dashboard & Progress Tracking

Status: **Backlog** · Priority: **P1** · Estimate: **L** · Depends on: **EPIC-01, EPIC-02, EPIC-08**

Description	The logged-in home. Shows the user's chosen path, overall course progression, module completion status, cookies earned, and lessons completed. The single place a learner goes to understand where they are.
Deliverables	• Dashboard page with progress overview • Chosen path display with option to change • Module completion grid (locked/in-progress/complete states) • Cookies earned summary • Overall course progression metric • Recently visited lesson shortcut
Acceptance Criteria	• Dashboard reflects real progress data from DB • Module states (locked/in-progress/complete) are accurate • Cookie balance matches exercise completion history • Page updates without full reload when progress changes
Out of Scope	• Leaderboards or social features • Detailed per-exercise analytics • Streak tracking (deferred)
Dependencies	• EPIC-01 — Design System • EPIC-02 — Auth • EPIC-08 — Cookie system (for balance display)
Initial Tasks	• Design dashboard layout • Build progress overview component • Build module status grid • Build cookie balance widget • Build overall progression metric • Connect to Supabase progress table • Build recently visited shortcut • Handle empty state (new user)

EPIC-07 — Module Engine

Status: **In progress** · Priority: **P0** · Estimate: **XL** · Depends on: **EPIC-01, EPIC-02, EPIC-11**

Description	The core learning interface. Three-tab lesson structure (Learn / Practice / Assess) with sticky navigation, lesson locking, progress tracking per lesson, and the sidebar lesson navigator. This is the primary product surface. v1.2 changes: (1) lesson unlocking is completion-based, not cookie-spend-based (plan §4.5); (2) the 'requires purchase' lock applies at track level (Option B), with a per-lesson freePreview exemption flag — lesson 1 of every paid module is playable without entitlement; (3) dependency corrected — this EPIC consumes the EPIC-11 content schema (done, so unblocked).
Deliverables	• Module page with sidebar lesson navigator • Three-tab layout per lesson (Learn / Practice / Assess) • Sticky topbar and tab bar • Lesson lock/unlock logic driven by previous-lesson completion • Track-level 'requires purchase' lock state with upgrade CTA • freePreview lesson exemption (lesson 1 of paid modules playable without entitlement) • Lesson completion detection and bonus payout • Mobile responsive layout with drawer navigation • Progress persistence per lesson
Acceptance Criteria	• Tab switching is instant with no layout shift • Assess tab locks until first exercise is attempted • Lesson N+1 unlocks when Lesson N is completed (no cookie deduction) • Purchase-locked lessons show upgrade CTA and never leak content (direct URL included) • freePreview lessons are fully playable without entitlement, with clear upgrade prompt at the end • Progress is saved to DB on each interaction • Mobile drawer opens and closes correctly • Forward reference tags render correctly in prose
Out of Scope	• Exercise engine (EPIC-08) • Quiz and assessment engine (EPIC-09) • Actual lesson content (EPIC-11 pipeline)
Dependencies	• EPIC-01 — Design System • EPIC-02 — Auth • EPIC-11 — Content schema (DONE — consumed here) • EPIC-06 — progress model alignment (non-blocking; keep schema consistent)
Initial Tasks	• Build module page layout • Build sidebar lesson navigator • Build tab system (Learn/Practice/Assess) • Implement sticky topbar • Implement sticky tab bar • Build lesson lock overlay (completion variant) • Build purchase lock overlay (track-level entitlement variant) • Implement freePreview lesson exemption • Implement completion-based unlock flow • Build mobile drawer navigation • Implement lesson completion detection • Connect progress to Supabase • Build forward reference tag component • Build Simply Put block component • Responsive QA

Resources

- Module 01 reference implementation (existing HTML)

EPIC-08 — Exercise & Cookie System

Status: **Backlog** · Priority: **P0** · Estimate: **XL** · Depends on: **EPIC-07**

Description	The exercise engine powering all interactive practice. Six exercise types, immediate per-answer feedback, hint system, and the revised cookie economy. v1.1 changes: (1) cookies are a reward/status currency — they are never required to progress; (2) IMPLEMENT ships here in an interim repo-based variant (starter repo + local test suite + self-check submission), upgraded to in-browser execution by EPIC-12; (3) before building, run a 30-line simulation of the earn/spend balance across real exercise counts per lesson to validate the numbers.
Deliverables	• ORDER exercise type • PREDICT exercise type • IDENTIFY exercise type • CLASSIFY exercise type • FIX exercise type • IMPLEMENT exercise type — interim repo-based variant • Immediate feedback with explanation per answer • Hint system (costs 5 cookies, available after first wrong attempt; never blocks progression) • Cookie earn and spend logic (reward model: answers, first-attempt bonus, completion bonuses) • Cookie balance persistence • Cookie toast notification • Exercise completion tracking
Acceptance Criteria	• All 6 exercise types render and function correctly • Hints are not available before first wrong attempt • A user at 0 cookies can still complete every exercise and lesson • Cookie balance updates immediately on earn/spend • Completing all exercises in a lesson triggers completion bonus • Exercise state persists if user navigates away and returns • Economy simulation shows no dead-end states for any plausible play pattern
Out of Scope	• In-browser code editor and live execution (EPIC-12) • Quiz questions (EPIC-09)
Dependencies	• EPIC-07 — Module Engine (exercises live inside lessons)
Initial Tasks	• Run cookie economy simulation against real lesson exercise counts • Build base ExerciseBlock component • Build ORDER exercise • Build PREDICT exercise • Build IDENTIFY exercise • Build CLASSIFY exercise • Build FIX exercise • Build interim IMPLEMENT exercise (repo brief + self-check submission) • Build feedback system • Build hint system • Implement cookie earn logic • Implement cookie spend logic (hints, early unlocks, War Room entries) • Build cookie toast • Build cookie balance hook • Persist cookie balance to Supabase • Build exercise completion tracker
Resources	• Module 01 reference implementation

EPIC-09 — Final Assessment & Module Completion

Status: **Backlog** · Priority: **P1** · Estimate: **L** · Depends on: **EPIC-07, EPIC-08**

Description	Lesson-level quizzes and the module final assessment. Grouped question submission, score reveal with per-question reasoning, retake capability, and module completion record stored to the user profile. Fast-follow after Phase 1 launch.
Deliverables	• Lesson-level quiz component (3-8 questions) • Scenario question type (flagged distinctly from MCQ) • Grouped submit with score reveal • Per-question reasoning displayed after submission • Retake functionality (best score persisted) • Module Final Assessment (8-10 questions, own nav slot) • Module completion record stored to DB • Cookie reward on quiz completion
Acceptance Criteria	• Quiz cannot be submitted with unanswered questions • Score reveal shows correct/incorrect per question with reasoning • Retake always shows fresh question order • Best score is preserved across retakes • Module Final unlocks only when all lessons visited, all quizzes submitted, all exercises completed or hints exhausted • Completion stored correctly to DB
Out of Scope	• Certificate PDF generation (deferred) • Peer review (deferred)
Dependencies	• EPIC-07 — Module Engine • EPIC-08 — Cookie system
Initial Tasks	• Build QuizBlock component • Build MCQ question type • Build Scenario question type • Implement grouped submit • Build score reveal with reasoning • Build retake flow • Build Module Final Assessment page • Implement unlock logic for Module Final • Persist completion to Supabase • Build completion celebration UI

EPIC-10 — Field Notes

Status: **Backlog** · Priority: **P2** · Estimate: **M** · Depends on: **EPIC-07, EPIC-09**

Description	A living reference section of the platform. Cheat sheets, DevTools guides, and quick-lookups for concepts covered in the course. Unlocks progressively as modules are completed. v1.1: early unlock purchasable with cookies (cookie spend sink in the revised economy).
Deliverables	• Field Notes section with navigation • Individual note pages (cheat sheet format) • Progressive unlock logic tied to module completion • Early unlock with cookies (–50) before module completion • Search within Field Notes • Locked state for notes not yet unlocked
Acceptance Criteria	• Notes unlock correctly when their linked module is completed • Early cookie unlock works and is clearly priced • Search returns relevant results • Notes render correctly on mobile • Locked notes show which module unlocks them
Out of Scope	• User-contributed notes • Comments or annotation system
Dependencies	• EPIC-07 — Module Engine • EPIC-09 — Module completion (unlock trigger)
Initial Tasks	• Design Field Notes layout • Build note page template • Build progressive unlock logic • Build cookie early-unlock flow • Build search functionality • Write initial notes for Module A01 • Build locked state UI

EPIC-11 — Tracks & Content Structure

Status: **Done** · Priority: **P0** · Estimate: **L** · Depends on: **None**

Description	The content data layer. Defines how modules, lessons, exercises, and quizzes are structured as data files (MDX for prose, JSON for exercises and quizzes). Schema designed for easy migration to a database later. v1.1: COMPLETE . Dependency direction corrected: this EPIC has no dependency on EPIC-07 — EPIC-07 consumes this schema. One follow-up: the content loader gains a server-side entitlement check hook when EPIC-14 lands.
Deliverables	• Folder and file structure for all content — SHIPPED • TypeScript types for Module, Lesson, Exercise, Quiz — SHIPPED • MDX pipeline for lesson prose — SHIPPED • Content loader utility (lib/content.ts) — SHIPPED • Migrated Module A01 content from HTML to content files — SHIPPED • tracks.json defining all tracks and module order — SHIPPED
Acceptance Criteria	• All content types are fully typed with no any — MET • Content loader reads files and returns typed data — MET • MDX prose renders correctly in the lesson view — MET • Adding a new lesson requires only adding files with no code changes — MET • Schema documented so a DB migration path is clear — MET
Out of Scope	• Actual content for modules A02-P09 (written alongside each module build) • Database migration (deferred) • Entitlement check hook (added with EPIC-14)
Dependencies	• None (corrected in v1.1 — previously listed EPIC-07 in error)
Initial Tasks	• All initial tasks complete. Follow-up: add entitlement check hook to lib/content.ts and freePreview flag to lesson schema when EPIC-14 lands.
Resources	• Next.js MDX docs • Module 01 reference implementation

EPIC-12 — In-Browser Code Editor & Test Runner

Status: **Backlog** · Priority: **P2** · Estimate: **XL** · Depends on: **EPIC-08**

Description	A sandboxed in-browser code editor for exercises that require writing and running real code. User submissions are run against a predefined test suite. Sandbox isolation is critical for security. v1.1: this EPIC upgrades the interim repo-based IMPLEMENT variant (EPIC-08) to fully in-browser execution; the open question of which exercise types need live execution is resolved during EPIC-08.
Deliverables	• Monaco or CodeMirror editor component • Sandboxed execution environment (iframe isolation) • Test runner that executes against user submission • Pass/fail feedback per test case • Reset to starter code functionality • Integration with Exercise system (replaces interim IMPLEMENT flow)
Acceptance Criteria	• User code cannot access or break the host application • Test results are displayed clearly per test case • Editor is usable on desktop (mobile is read-only) • Syntax highlighting works for JavaScript and TypeScript • Execution has a timeout to prevent infinite loops
Out of Scope	• Server-side code execution • Support for languages other than JS and TS • Saving code snippets
Dependencies	• EPIC-08 — Exercise system (this extends it)
Initial Tasks	• Evaluate Monaco vs CodeMirror • Build editor component • Implement iframe sandbox • Build test runner • Build test result display • Implement execution timeout • Implement reset functionality • Integrate with ExerciseBlock (swap interim IMPLEMENT) • Security review
Resources	• Monaco Editor docs • CodeMirror docs • FreeCodeCamp open source (reference)

EPIC-13 — Interview Accelerator

Status: **Backlog** · Priority: **P2** · Estimate: **XL** · Depends on: **EPIC-07, EPIC-11**

Description	Interview preparation content built incrementally alongside each track. Each track gets its own dedicated interview section covering the specific skills, knowledge, and question types relevant to that career level. Cross-references existing module content rather than duplicating it. v1.1 strategic note: remains P2 in build order, but IP-01 is the most monetizable asset in the catalog ('pass the interview' is the most concrete outcome). If early-access revenue underperforms, IP-01 is first in line for promotion to a standalone paid product.
Deliverables	• IP-01 Junior/Mid Interview Prep (ships after Foundations and Professional tracks) • IP-02 Senior/Staff Interview Prep (ships after Advanced Track) • Interview Guide content type (topic intro, module cross-reference, practice question, model answer) • Whiteboard-style and verbal explanation exercise format • Time-pressure exercise variant with visible countdown • Model answers with scoring rubric • Interview Accelerator as a distinct section in the platform navigation
Acceptance Criteria	• Every interview question references at least one existing module lesson • Cross-reference links navigate correctly to the relevant lesson • Time-pressure exercises have a visible countdown • Model answers are only revealed after submission • Content is clearly separated by track and level so learners do not land in the wrong tier • Works correctly on mobile
Out of Scope	• Mock interview scheduling or live sessions • AI-powered interview simulation (deferred, strong candidate for a future EPIC) • Company-specific interview guides • Salary negotiation content
Dependencies	• EPIC-07 — Module Engine (Interview Guides live inside the platform shell) • EPIC-11 — Tracks and Content Structure (new Interview Guide content type needed) • Advanced Track A01-A06 must exist before IP-02 ships • Foundations and Professional tracks must exist before IP-01 ships
Initial Tasks	• Define Interview Guide content type and schema • Build Interview Guide page template • Build cross-reference component linking to existing lessons • Build whiteboard exercise format • Build time-pressure exercise variant • Build model answer reveal component • Write IP-02 content for Advanced Track (V8, memory, rendering, performance questions) • Build Interview Accelerator navigation section • Write IP-01 content incrementally as Foundations and Professional tracks complete
Resources	• Existing module content as source for all cross-references • Blind 75 and NeetCode for question format reference • Greatfrontend and BFE.dev frontend interview question banks

EPIC-14 — Payments & Entitlements [NEW]

Status: **Backlog** · Priority: **P0** · Estimate: **M** · Depends on: **EPIC-02**

Description	The revenue layer: checkout, entitlements, and the free/paid boundary. Boundary DECIDED (v1.2) — Option B: Foundations track completely free; Professional + Advanced paid. Entitlements at track granularity (no per-module rows), plus a per-lesson freePreview exemption (lesson 1 of every paid module). Uses a Merchant of Record (Paddle or Lemon Squeezy) so EU VAT per buyer country, invoicing, and tax filing are handled by the provider — do not build raw Stripe for B2C in the EU. Entitlements stored in Supabase and enforced server-side in the content loader; client state is presentation only. Phase 1 ships a single product: Professional track early access (price locked for pre-order customers).
Deliverables	• Boundary implemented per Option B: Foundations free, Professional + Advanced track-level entitlements, freePreview flag on lesson 1 of paid modules • Merchant of Record account, product catalog, and checkout integration • Webhook handler: purchase → entitlement row in Supabase • entitlements table (user_id, track_id, source, granted_at, revoked_at) • Server-side entitlement check in content loader (EPIC-11 hook) • Purchase success / failure pages • Customer portal link (receipts, refunds via MoR) • Early-access product live: Professional track (early-access pricing honoured for pre-orders)
Acceptance Criteria	• A test purchase grants access to paid content within seconds (webhook path verified) • Refund via MoR revokes entitlement automatically • Paid content is unreachable without entitlement, including via direct URL and API • All Foundations content and freePreview lessons are reachable without any purchase • Checkout works from EU countries with correct VAT displayed • Webhook handler is idempotent (duplicate events do not duplicate entitlements)
Out of Scope	• Subscriptions (one-time purchases first; revisit post-launch) • Discount codes and affiliates (MoR supports later) • Team / B2B licensing • Custom invoicing
Dependencies	• EPIC-02 — Auth (purchases attach to a user) • Informs EPIC-04 (entitlement badges) and EPIC-07 (purchase lock state)
Initial Tasks	• Evaluate Paddle vs Lemon Squeezy (fees, payout to DE, API ergonomics) • Create MoR account and Professional early-access product • Design track-level entitlements schema in Supabase • Add freePreview flag to lesson schema (EPIC-11 follow-up) • Implement checkout flow (hosted checkout preferred) • Implement webhook handler with idempotency • Add entitlement check to content loader • Build success/failure pages • Wire 'requires purchase' CTA in Module Engine • End-to-end test: purchase, access, refund, revoke
Resources	• Paddle docs • Lemon Squeezy docs • Supabase RLS docs (defense in depth on entitlements)

EPIC-15 — Launch Readiness — Email, Analytics, Legal [NEW]

Status: **Backlog** · Priority: **P1** · Estimate: **M** · Depends on: **EPIC-02, EPIC-14**

Description	NEW in v1.1. The unglamorous layer every paid product needs. Three workstreams: (1) transactional email — required the moment money moves; (2) product analytics — a learning funnel cannot be tuned blind (where do learners drop off inside a lesson?); (3) legal pages — operating from Germany, Impressum is mandatory, plus privacy policy (GDPR), terms of service, and withdrawal-right disclosure for digital goods.
Deliverables	• Transactional email provider integrated (Resend or Postmark): welcome, receipt handoff, purchase confirmation • Privacy-friendly analytics (PostHog EU or Plausible + custom events) • Core event taxonomy: signup, onboarding complete, lesson start/complete, exercise attempt/solve, hint reveal, quiz submit, checkout start/complete • Funnel dashboards: visit → signup → first lesson → purchase; per-lesson drop-off • Impressum page • Privacy policy (GDPR) • Terms of service • Digital goods withdrawal-right consent at checkout (EU consumer law)
Acceptance Criteria	• Purchase confirmation email arrives within one minute of checkout • All core events fire and are visible in dashboards • Per-lesson drop-off is queryable • Impressum reachable from every page footer • Cookie/consent handling compliant with analytics choice • Withdrawal-right consent captured before content access
Out of Scope	• Marketing automation / drip campaigns • A/B testing infrastructure • Cookie banner beyond what the chosen analytics requires
Dependencies	• EPIC-02 — Auth (user lifecycle events) • EPIC-14 — Payments (receipts, checkout events)
Initial Tasks	• Choose email provider and set up domain auth (SPF/DKIM) • Implement welcome + purchase confirmation emails • Choose analytics (PostHog EU vs Plausible) and integrate • Define and implement event taxonomy • Build funnel dashboards • Write Impressum • Write privacy policy • Write terms of service • Add withdrawal-right consent to checkout • Footer links on all pages
Resources	• Resend / Postmark docs • PostHog EU cloud docs • IT-Recht / eRecht24 Impressum and privacy generators (review before publishing)

EPIC-16 — The War Room [NEW]

Status: **Backlog** · Priority: **P2** · Estimate: **L** · Depends on: **EPIC-08, EPIC-09, EPIC-14**

Description	NEW in v1.4. Timed debugging challenges framed as production incidents, for paid-tier users. Three-screen lifecycle specified in the v1.3 design mockups: briefing (scenario, materials, stakes — timer starts only on explicit entry), investigation (symptoms + console log + network timeline alongside a multi-file source viewer; two-step resolution: diagnosis on selected lines, then choice of fix), post-mortem (itemized payout, full explanation, 'what you could have noticed earlier'). No hints by design — the clues live in the materials, as in a real incident. Built as a thin runner on the EPIC-08 engine: each challenge is a timed IDENTIFY→FIX chain with structured payout, so no live code execution is required; EPIC-12 later upgrades the fix step to written code (not a prerequisite).
Deliverables	• Challenge lobby with difficulty, duration, reward and locked states • Briefing screen: scenario, materials list, entry fee and reward stakes, explicit-start timer • Investigation screen: materials panel (symptoms, console log, network timeline) • Multi-file read-only source viewer with selectable lines • Two-step challenge runner (diagnosis → fix) built on ExerciseBlock • Countdown timer with thresholds (amber default, red under 2 minutes, plus textual state) • Entry fee (–25), refund-on-solve, itemized payout (entry refund / diagnosis / fix / time bonus) • Abandon and timeout flows (entry fee forfeited) • Post-mortem screen: payout breakdown, explanation, 'what you could have noticed earlier', lesson cross-reference • Challenge content schema (extends EPIC-11 types) and 3 launch challenges
Acceptance Criteria	• Timer starts only on explicit entry confirmation, never on page view • Entry fee is deducted on entry, refunded plus reward on solve, forfeited on abandon or timeout • No hint UI is rendered anywhere in a war room challenge • Correct diagnosis with wrong fix still pays the diagnosis share • Line selection is required before diagnosis options are enabled • Post-mortem is shown for solved, failed and timed-out runs alike • Locked challenges display their unlock requirement (module completion) • Challenges are unreachable without paid entitlement (track-level check) • Timer state is conveyed by text as well as color; reduced motion respected
Out of Scope	• Live code execution for the fix step (EPIC-12 upgrade) • Leaderboards and time rankings • Multiplayer or co-op incidents • User-submitted challenges
Dependencies	• EPIC-08 — exercise engine (IDENTIFY/FIX blocks, cookie earn/spend logic) • EPIC-09 — module completion (unlock gating for challenges) • EPIC-14 — entitlements (war room is paid tier, Option B) • EPIC-11 — content schema extension for challenge files

Initial Tasks	• Define challenge content schema (scenario, materials, files, steps, payout) • Build lobby challenge cards incl. locked state • Build briefing screen • Build materials panel components (console log, network timeline) • Build multi-file source viewer with selectable lines • Implement two-step diagnosis→fix runner on ExerciseBlock • Implement countdown timer with thresholds and textual state • Implement entry fee, refund and itemized payout logic • Build abandon and timeout flows • Build post-mortem screen • Author 3 launch challenges (medium, hard, hard) • Accessibility pass (timer, keyboard line selection, reduced motion)
Resources	• War room screen specs (design mockups v1.3) • EPIC-08 ExerciseBlock implementation